PROJECT REPORT

Address Book Management System

Developed in: C Programming Language

Architecture: Command-Line Interface (CLI)

Data Storage: Comma-Separated Values (CSV)

1. INTRODUCTION

The Address Book Management System is a command-line application developed in C. It serves as a digital directory for storing and managing contact information, including names, phone numbers, and email addresses. The primary objective of this project is to provide a lightweight, efficient, and persistent solution for personal contact management while demonstrating fundamental concepts of structured programming, memory management, and file I/O operations.

2. SYSTEM ARCHITECTURE & MODULES

The system is designed with a modular architecture, separating the user interface, business logic, and data persistence layers into distinct source files.

2.1. Main Controller (main.c)

This module acts as the entry point of the application. It initializes the `AddressBook` structure and presents a continuous, menu-driven interface to the user using a `do-while` loop and `switch` statements, routing user choices to the appropriate logical functions.

2.2. Core Logic (contact.c & contact.h)

This module handles all CRUD (Create, Read, Update, Delete) operations. It includes:

- **Validation Algorithms:** Ensures data integrity before insertion. Phone numbers are validated for exactly 10 digits, and emails are checked for the presence of standard `@` and `.com` domains. It also enforces unique entries for emails and phone numbers.
- **Search Engine:** Implements a linear search utilizing `strcasestr()` to allow for case-insensitive substring matching, significantly improving the user experience over exact-match requirements.
- **Sorting Mechanism:** Utilizes a Bubble Sort algorithm to organize the contact list dynamically by Name, Phone Number, or Email Address upon user request.

2.3. Data Persistence (file.c & file.h)

To ensure data is not lost when the program terminates, this module serializes the application state to a disk file named `contacts.csv`. It parses the CSV upon initialization using `fscanf` with regex-like specifiers (`%[^,]`) to reconstruct the struct array, and uses `fprintf` to write the data back to disk upon exit, complete with a visual terminal progress bar.

3. DATA STRUCTURES

The system leverages a statically allocated array of structures to maintain optimal CPU cache locality for rapid sorting and searching.

```
typedef struct {
    char name[50];
    char phone[20];
    char email[50];
} Contact;

typedef struct {
    Contact contacts[MAX_CONTACTS];
    int contactCount;
} AddressBook;
```

4. KEY FEATURES & OPERATIONS

Feature	Description & Implementation
Create Contact	Accepts user input, passes it through robust string validation loops, and appends it to the `contacts` array if unique.
Edit / Delete	Retrieves a contact via a search index, allowing targeted modification of fields or removal via a left-shift array operation (O(n) complexity).
Sorting	Provides a formatted tabular view of all contacts, organized alphabetically or numerically using an in-place Bubble Sort.

5. LIMITATIONS AND FUTURE ENHANCEMENTS

While the current implementation is highly functional, there are areas identified for future scalability:

- **Static Memory Allocation:** The system is currently capped at `MAX_CONTACTS 100`. Transitioning from a static array of structs to a Dynamically Allocated Linked List would allow for infinite scaling limited only by system heap memory.
- **State Management:** The search functionality relies on a global integer array (`int arr[100], size=0;`) to track indices. Refactoring this to pass a dynamically allocated array pointer would make the program strictly thread-safe.
- **Sorting Efficiency:** The current Bubble Sort operates at $O(n^2)$ time complexity. For larger datasets, implementing the standard library's `qsort()` (O(n log n)) would yield significant performance improvements.

6. CONCLUSION

The Address Book Management System successfully demonstrates the practical application of C programming fundamentals. By successfully integrating complex string manipulation, custom struct data typing, algorithm implementation, and persistent file I/O, the project serves as a robust foundational software utility and a strong proof of concept for low-level application design.